

Cryptagent evaluation report

1. What “50 not reviewed” means

The agent has five requirements per submitted code:

1. Functional correctness (F01)
2. Tampering rejection (F02)
3. Authentication before output (F03)
4. Key and nonce handling (F04)
5. Input boundaries (F05)

There were ten code submissions. During the recorded run, the model completed none of the nine live reviews and the separate missing-key control also did not reach the model. Therefore every requirement in every report is marked **not reviewed**:

10 codes × 5 requirements = 50 not-reviewed requirement entries.

These are not 50 test programs, vulnerabilities, or failures in the code. They mean the agent had no accepted source-review evidence for any requirement.

2. Agent working and structure

Cryptagent is a Windows-local web application that reviews C++ code against one fixed midterm profile:

Setting	Current value
Cryptographic integration	AES-128-GCM
Native API target	Windows CNG
Attacker	Message tampering
LLM	GPT-6 Astra
Reasoning	Medium
Input limits	64 KiB and 2,000 lines
Generation	Disabled
Learning	Disabled
Native execution	Not connected
Formal verification	Not connected

The profile requires a 16-byte AES key, a 12-byte nonce and a 16-byte tag. It asks whether valid inputs work, whether tampering with ciphertext/nonce/tag/AAD is rejected, whether unauthenticated plaintext is withheld, whether key/nonce handling is safe, and whether length/buffer boundaries are handled.

Workflow

```
C++ code entered in local page
↓
Request/profile/size validation
↓
Source hash + small deterministic pattern scan
↓
OpenAI model access check
↓
GPT-6 Astra structured source review
↓
Schema and exact source-quote validation
↓
Overall assessment + five requirement assessments
↓
Saved evaluation response
```

The model receives trusted reviewer instructions and profile rules, followed by the code as numbered untrusted lines. It must return exactly one scope result, five per-requirement assessments and at most eight source-quoted findings. The backend rejects invalid IDs, duplicate checks, inconsistent result combinations and findings whose quotations cannot be found in the submitted source.

The app can report: **Potential issues**, **No issues identified**, **Insufficient context**, **Outside scope**, or **Review unavailable**. The final category has highest priority whenever no completed model review was accepted.

Important boundary: this is currently a source-review agent. It does not compile the pasted C++, execute it, run tampering tests, or formally prove AES-GCM properties. “No issues identified” would be a review observation, not proof that code is secure or deployable.

3. Recorded evaluation data

The dataset includes ten saved C++ fixtures, ten saved JSON responses and a manifest describing the intended outcome of each fixture. All ten normalized source hashes match their recorded responses, so the input-to-response linkage is intact.

Metric	Result
Code fixtures	10
Unique source hashes	9
Live review attempts	9
Accepted live model reviews	0
Missing-key control	1
Rate/quota errors	9
Requirement assessments returned as not reviewed	50
Accepted model findings	0
Native execution runs	0
Formal-verification runs	0

Cases 07 and 10 contain the same baseline source. Case 10 changes only the environment: it removes the API key in a separate process. It is a service-control case, not a separate C++ implementation.

4. Results for the ten tested codes

The “intended outcome” is the controlled test design. It is not a claim that the LLM produced that assessment. “Recorded output” is what the agent actually returned.

Case	Code type	Intended outcome	Recorded output
01	Ignored authentication failure	Potential issues	Review unavailable
02	Nonce reuse	Potential issues	Review unavailable
03	Short authentication tag	Potential issues	Review unavailable
04	Associated data ignored	Potential issues	Review unavailable
05	Output before authentication check	Potential issues	Review unavailable
06	Incorrect output capacity	Potential issues	Review unavailable
07	Complete session integration	No issues identified	Review unavailable
08	Missing implementation	Insufficient context	Review unavailable
09	Unrelated sorting	Outside scope	Review unavailable
10	Service unavailable control	Review unavailable	Review unavailable

Case 01: Ignored authentication failure

Code design. The failure branch is replaced by `if (false)`. The function can release output and return true without acting on an unsuccessful BCryptDecrypt status.

Recorded output. Review unavailable.

Why the agent returned it. OpenAI rate/quota error; no LLM assessment was accepted. The local report contained no accepted findings and five not_reviewed entries. This is an infrastructure result, not the LLM's judgment of the code.

What to verify next. Tamper with the authentication tag and require false plus an empty/wiped caller output.

Case 02: Nonce reuse

Code design. `sequence = 0` before each encryption makes the encoded nonce repeat under the same session key.

Recorded output. Review unavailable.

Why the agent returned it. OpenAI rate/quota error; no LLM assessment was accepted. The local report contained no accepted findings and five not_reviewed entries. This is an infrastructure result, not the LLM's judgment of the code.

What to verify next. Encrypt two messages under one SessionCipher instance and require different nonces.

Case 03: Short authentication tag

Code design. The profile requires a 16-byte tag, but the CNG calls pass `cbTag = 8`. This is a profile mismatch.

Recorded output. Review unavailable.

Why the agent returned it. OpenAI rate/quota error; no LLM assessment was accepted. The local report contained no accepted findings and five not_reviewed entries. This is an infrastructure result, not the LLM's judgment of the code.

What to verify next. Verify provider behavior and test a changed byte in the second half of the 16-byte packet tag.

Case 04: Associated data ignored

Code design. The API accepts associated data but passes nullptr and length zero to CNG on both paths. Nonempty caller AAD is therefore not authenticated.

Recorded output. Review unavailable.

Why the agent returned it. OpenAI rate/quota error; no LLM assessment was accepted. The local report contained no accepted findings and five not_reviewed entries. This is an infrastructure result, not the LLM's judgment of the code.

What to verify next. Encrypt with nonempty AAD; alter only AAD during decrypt and require rejection.

Case 05: Output before authentication check

Code design. Temporary output is copied to caller-visible output before the status check. The failure branch does not wipe the caller output.

Recorded output. Review unavailable.

Why the agent returned it. OpenAI rate/quota error; no LLM assessment was accepted. The local report contained no accepted findings and five not_reviewed entries. This is an infrastructure result, not the LLM's judgment of the code.

What to verify next. Submit a tampered packet and inspect both return value and caller-visible output.

Case 06: Incorrect output capacity

Code design. Eight bytes are allocated, but ciphertext size is supplied as CNG's output capacity. Longer input can advertise writable space beyond the allocation.

Recorded output. Review unavailable.

Why the agent returned it. OpenAI rate/quota error; no LLM assessment was accepted. The local report contained no accepted findings and five not_reviewed entries. This is an infrastructure result, not the LLM's judgment of the code.

What to verify next. Run boundary lengths 1, 8, 9 and a larger permitted input inside an isolated worker.

Case 07: Complete session integration

Code design. Positive-control baseline: 16-byte key, 12-byte nonce, 16-byte tag, AAD, locked counter, output released after successful decrypt status. It remains an uncompiled, untested design.

Recorded output. Review unavailable.

Why the agent returned it. OpenAI rate/quota error; no LLM assessment was accepted. The local report contained no accepted findings and five not_reviewed entries. This is an infrastructure result, not the LLM's judgment of the code.

What to verify next. Compile and independently review it before treating it as a positive control; then run reference, tampering and boundary tests.

Case 08: Missing implementation

Code design. Only function declarations are supplied. Key generation, nonce lifecycle, tag checks and buffer handling are absent.

Recorded output. Review unavailable.

Why the agent returned it. OpenAI rate/quota error; no LLM assessment was accepted. The local report contained no accepted findings and five not_reviewed entries. This is an infrastructure result, not the LLM's judgment of the code.

What to verify next. Require a completed review to name the missing implementation evidence without inventing it.

Case 09: Unrelated sorting

Code design. A small integer-sorting function has no AES-GCM or Windows CNG integration. A completed review should classify it as outside the fixed profile.

Recorded output. Review unavailable.

Why the agent returned it. OpenAI rate/quota error; no LLM assessment was accepted. The local report contained no accepted findings and five not_reviewed entries. This is an infrastructure result, not the LLM's judgment of the code.

What to verify next. Require outside scope, five not-applicable checks and no cryptographic findings in a completed review.

Case 10: Service unavailable control

Code design. Same source as case 07, but evaluated in a separate process with OPENAI_API_KEY removed. This tests missing-key handling, not C++ behavior.

Recorded output. Review unavailable.

Why the agent returned it. The local credential gate returned missing_key; the model was not called. The local report contained no accepted findings and five not_reviewed entries. This is an infrastructure result, not the LLM's judgment of the code.

What to verify next. Require review unavailable, model state missing_key, sourceSent false, and no execution/formal evidence.

5. Analysis of the run

The experiment measured availability, not model detection quality

All nine live cases produced the same rate/quota error path. This included both large CNG integrations and the seven-line sorting function. The common failure source is the OpenAI request path, not a shared code property.

The local application returned HTTP 200 with a structured application report. That means the local server worked; it does **not** mean the model completed its review. The report explicitly recorded model.state: error, an OpenAI rate/quota message, and sourceSent: true for cases 01–09. The last field indicates the request was attempted; it does not prove completed inference or establish token usage.

Case 10 correctly tested a different condition: its separate process had no API key. The client detected this locally before a model request, returned missing_key, and set sourceSent: false.

No model accuracy metric can be reported

There are zero completed model outputs, zero accepted model findings and zero reviewed requirement assessments. Therefore accuracy, precision, recall, false-positive rate and F1 cannot be calculated. The one intended-label match, case 10, measures only missing-key handling and must not be presented as model accuracy.

The deterministic scan missed the six planned faults

The local pattern scanner returned zero observations for all six controlled issue fixtures. These faults are semantic:

- control flow that makes an error branch unreachable,
- cross-call nonce state reset,
- an authentication tag API length mismatch,

- omitted associated data,
- output release before a status decision,
- allocation/capacity disagreement.

The scanner mostly searches for simple textual forms such as ignored decrypt calls, zero initialization near variables named nonce/iv, sensitive printing and ECB identifiers. It does not do data-flow, control-flow, API-argument or state-lifecycle analysis. Its result in this run is **0 of 6 intentional mutation cases flagged**, but that is not an LLM recall score.

The access indicator needs careful interpretation

“Access confirmed” checks whether the selected model can be retrieved through the account. It did not prove that the Responses API had quota available. This run shows why the UI should eventually show separate states for model access, last completed review and available quota/rate-limit status.

Timing is failure-handling latency, not inference latency

The nine live records took roughly 0.75 to 2.96 seconds from runner timestamp to report creation, with a mean around 1.43 seconds. These are error-path timings. They do not represent GPT-6 Astra’s successful review speed.

6. Future scope

1. Restore API availability first. Run one minimal source review and confirm a completed structured response before launching another batch.
2. Preserve this run as a failed-service dataset. Create a new run ID or an explicit retry-failed mode; the current runner skips existing response files.
3. Repeat the nine code-review fixtures. Compare actual results with independently reviewed ground truth, not just intended labels.
4. Add a real Windows-isolated execution worker with a defined C++ adapter. It should compile complete submissions, run reference vectors, mutate ciphertext/nonce/tag/AAD, inspect output gating and exercise boundaries.
5. Add bounded formal checks for explicit wrapper properties. A tool such as CBMC can be used for bounded C/C++ assertions, but its assumptions, loop bounds, Windows CNG model and proof logs must be visible. A bounded proof of wrapper behavior is not a proof that the external CNG provider or all hardware behavior is secure.
6. Keep LLM findings, executable evidence and formal evidence separate. A model should never independently mark a codebase deployable.
7. Begin reviewed learning only after a completed and independently reviewed test phase. Do not learn from rate-limit errors or raw model claims.

7. Conclusion

Cryptagent’s present architecture has a clear fixed scope, structured review contract, input hashing and an honest “review unavailable” route. The recorded ten-case batch verifies the availability-failure path and preserves the test inputs correctly. It does **not** yet demonstrate that the LLM detects the six planned cryptographic integration faults, recognizes the intended baseline, or formally verifies C++ code.

The immediate next milestone is a successful, reproducible source-review batch. Execution and formal verification should then be added as distinct evidence layers, not substituted with an LLM label.

Appendix: evidence files

- Raw evaluation records
- Original test-run summary
- Test intentions
- Inputs
- Individual saved responses